

EMI-3 - António Cruz, Bruno Santos, Pedro Miranda, Ricardo Kayseller.

1. Introduction

This report presents the first incremental delivery of **noted**, a collaborative, web-based MLOps workbench that unifies the full machine learning lifecycle - data versioning, experiment tracking, configuration management, pipeline orchestration, and model serving - into a single, integrated environment. Rather than aggregating existing tool UIs, noted provides purpose-built views that communicate with backend services through their APIs. The underlying tools - DVC, MLflow, Hydra, Airflow, MinIO - remain the engines; noted is the cockpit.

The platform runs as a Docker Compose stack comprising 13 containers: the main noted application (FastAPI backend + vanilla ES6 frontend), MLflow, MinIO, PostgreSQL, Redis, a full Apache Airflow cluster, and an nginx reverse proxy routing all services under a single origin. All artifacts are standard and portable - notebooks are .ipynb, MLflow runs use the standard API, DVC pointer files are standard YAML - ensuring zero vendor lock-in.

For this Tutorial #1, we apply noted to a weather forecasting problem using the Jena Climate dataset, demonstrating the foundational MLOps layers: data versioning (DVC + MinIO) and experiment tracking (MLflow). The accompanying notebook shows the complete workflow: data ingestion, DVC tracking, preprocessing, baseline model training with strict data lineage logged to MLflow.

2. Dataset Description

The Jena Climate dataset is a multivariate meteorological time series recorded by the Max Planck Institute for Biogeochemistry in Jena, Germany. It contains atmospheric measurements collected every 10 minutes from January 2009 to December 2016, totaling approximately 420,000 observations across 14 meteorological variables plus a timestamp.

The raw dataset is managed via DVC with a MinIO remote storage backend. DVC assigns a cryptographic MD5 hash to each tracked file, and this hash is logged as an MLflow parameter in every experiment run, establishing strict data lineage. Preprocessing includes resampling to hourly frequency, replacement of erroneous wind speed values (-9999.0), duplicate removal, and temporal splitting into train/validation/test sets.

3. Machine Learning Problem

The forecasting task is formulated as a multivariate-input, univariate-output, multi-step regression problem: given a sliding window of past meteorological observations, predict the air temperature (T) for the next hours ahead.

For this Tutorial #1, we implement a linear regression baseline to validate the end-to-end MLOps pipeline: data ingestion, DVC tracking, preprocessing, training, and MLflow experiment logging with strict data lineage. Subsequent deliveries will introduce GRU and PatchTST architectures with Optuna hyperparameter optimization, multi-step forecasting (120h input window, 24h horizon), and systematic comparison across architectures. The final implementation will be based on TAAP MP3.

4. DVC, Hydra, and MLflow Integration

This is the core contribution of Tutorial #1. The accompanying notebook demonstrates the full data and configuration lineage chain:

1. **Data Ingestion** - Download the Jena Climate CSV from the TensorFlow dataset repository.
2. **DVC Tracking** - `dvc add` creates a `.dvc` pointer file with the MD5 hash; `dvc push` syncs to MinIO.
3. **Data Lineage** - The DVC hash is read from the `.dvc` file and logged to MLflow as both a parameter (`dvc_data_hash`) and a tag (`dvc.data_hash`), linking every experiment run to the exact dataset version.
4. **Hydra Configuration** - Experiment parameters (data paths, feature selection, split ratios, model type) are defined in a centralized `config/config.yaml` file, loaded via Hydra. The resolved configuration is hashed and logged to MLflow as a parameter (`hydra_config_hash`), a tag (`hydra.config_hash`), and saved as an artifact (`hydra_config.yaml`) - establishing configuration lineage alongside data lineage.
5. **Experiment Tracking** - A baseline model is trained within an `mlflow.start_run()` context, logging all parameters, metrics, and both lineage hashes (data + config)
6. **Verification** - The logged lineage is queried back from MLflow to confirm traceability of both data version and configuration.

Within noted, this workflow is further enhanced by platform features built for this delivery: a Source Control panel with Git and DVC sections side by side, VS Code-style tree decorations for DVC-tracked files, context menu tracking (“Track with DVC”), an experiments browser with run detail panels, and automatic `MLFLOW_TRACKING_URI` injection into every kernel.

5. Project Structure

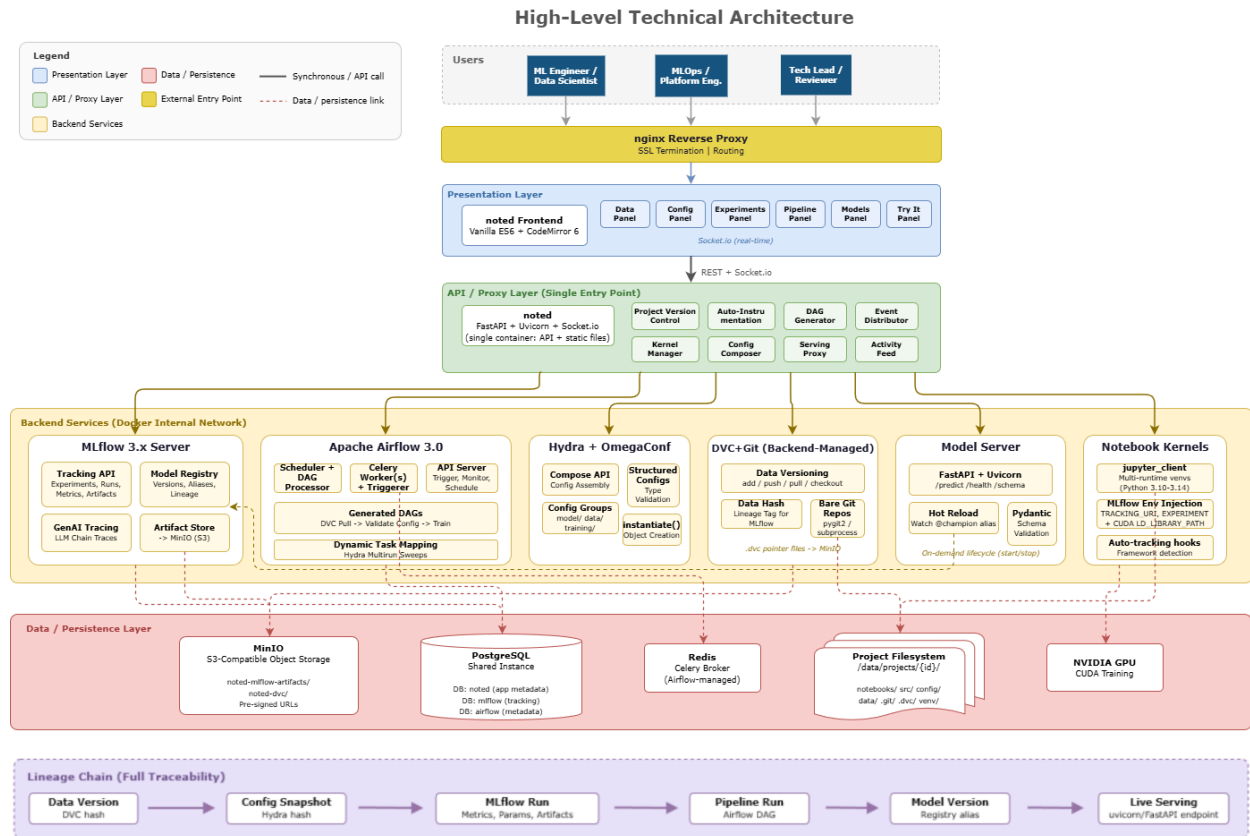
The repository follows a modular layout aligned with the MLOps lifecycle, separating concerns across clearly defined directories:

jena_weather/	
config/	Hydra configuration files (experiment parameters, model settings)
data/	DVC-managed datasets (raw CSV, <code>.dvc</code> pointer files)
dags/	Airflow DAG definitions (pipeline orchestration)
models/	Trained model artifacts (MLflow-tracked)
notebooks/	Interactive exploration and tutorial notebooks
report/	Written deliverables and documentation
src/	
ingestion/	Data download and ingestion scripts
training/	Model training and hyperparameter optimization
evaluation/	Model evaluation and metric computation
serving/	FastAPI/LitServe model serving endpoints
README.md	

For this Tutorial #1, the active directories are `data/`, `config/`, `notebooks/`, and `report/`. The `src/` modules, `dags/`, and `models/` directories are scaffolded for subsequent deliveries where notebook logic will be extracted into reusable scripts, orchestrated via Airflow DAGs, and model artifacts will be registered in the MLflow Model Registry.

6. Architecture and Lineage Diagram

The following diagram illustrates noted’s high-level technical architecture. The bottom row shows the full lineage chain: Data Version (DVC hash) to Config Snapshot to ML Run (MLflow) to Pipeline Run (Airflow) to Model Version (Registry) to Live Serving.



7. Next Steps

Upcoming deliveries will extend noted with:

- Tutorial #2** - Expanded Hydra configuration management (config groups for model architectures, CLI override builder) and Airflow pipeline orchestration (DAG definition, parameterized triggers, live monitoring)
- Final Delivery** - MLflow Model Registry integration, FastAPI/LitServe model serving, and a standalone prediction frontend